Tema 3

Utilización de objetos. Objetos nativos del lenguaje.



ÍNDICE

01 Utilización de objetos. Objetos nativos del lenguaje.

02 Interacción con el navegador. Objetos predefinidos asociados.

03 Generación de texto y elementos HTML desde código.

04 Gestión de la apariencia de la ventana.

05 Creación de nuevas ventanas. Comunicación entre ventanas.

06 Interacción con el usuario.

07 Mecanismos del navegador para el almacenamiento y recuperación de información.

Clases y objetos

- La programación orientada a objetos agrupa datos (**propiedades**) y comportamiento (**métodos**) en una misma estructura.
- Una **clase** es la plantilla; un **objeto** es un ejemplar concreto creado a partir de ella.
- **Instanciar** es crear un objeto con **new**. A sus miembros se accede con el operador punto.
- Por convención, las clases se nombran en PascalCase.

```
  constructor(nombre, telefono) {
    this.nombre = nombre;        // propiedades
    this.telefono = telefono;
  }
  saludar() {                    // método
    return `Hola, soy ${this.nombre}`;
  }
}

const ana = new Contacto("Ana", "600111222");
ana.nombre;       // "Ana"
ana.saludar();    // "Hola, soy Ana"
```

Herencia, encapsulación y polimorfismo

Herencia

Una clase obtiene las propiedades y métodos de otra con extends. JavaScript solo admite herencia simple; la múltiple se simula con mixins.

Encapsulación

Se ocultan los detalles internos y solo se expone lo necesario. En JS, con campos privados (#) y getters.

Polimorfismo

Objetos distintos responden al mismo método, cada uno a su manera.

Objetos nativos del lenguaje

Objeto	Para qué	Ejemplo
String	Cadenas de texto	"hola".toUpperCase()
Number	Números	Number.parseFloat("3.5")
Boolean	Valores lógicos	Boolean(0) → false
Array	Listas de valores	[3, 1, 2].sort()
Date	Fechas y horas	new Date().getFullYear()
Math	Funciones matemáticas	Math.round(2.5)
RegExp	Patrones de texto	/^\d{5}$/.test("46001")
Function	Funciones (también son objetos)	typeof alert → "function"
JSON	Convertir a y desde texto JSON	JSON.parse('{"a": 1}')

Objetos nativos del lenguaje: Date

- El objeto **Date** guarda un instante como los milisegundos transcurridos desde el 1 de enero de 1970 (UTC).
- Hay que tener en cuenta:
  - **Los meses empiezan en 0:** enero es el mes 0, pero los días empiezan por 1.
  - **Las fechas se desbordan en lugar de fallar:** el 32 de enero se convierte en el 1 de febrero.
  - **El formato del texto cambia la zona horaria:** **new Date("2026-10-30")** se interpreta en UTC, mientras que **new Date("2026-10-30T00:00")** se interpreta en la hora local.
  - **Son mutables:** **date.setDate(...)** modifica el objeto.

Objetos nativos del lenguaje: Date

➔ Algunos ejemplos:

Instrucción	**Dato obtenido**
getFullYear()	Año
getMonth()	Mes
getDate()	Día
getDay()	Día semana
getHours()	Hora
getMinutes()	Minutos
getSeconds()	Segundos
getMilliseconds()	Milisegundos
getTime()	Timestamp
getTimezoneOffset()	Timezone

Objetos nativos del lenguaje: String

- Al crear una cadena, esta se convierte de manera implícita en un objeto String, con sus propiedades y métodos predefinidos.
- Los métodos de manipulación de cadenas no modifican al objeto actual, sino que devuelven el objeto resultante de aplicar la modificación:
  - Cuando hacemos **name.toUpperCase()** sabemos que **name** no cambia.
  - Podemos guardar el resultado de la siguiente manera: **name = name.toUpperCase()**

Objetos nativos del lenguaje: String

➔ Algunos ejemplos:

Instrucción	**Dato obtenido**
toLowerCase() / toUpperCase()	Devuelve la cadena convertida a minúsculas/mayúsculas.
concat(cadena)	Devuelve el objeto con el valor de cadena concatenado al final.
charAt(posicion)	Devuelve el carácter que se encuentre en la posición solicitada.
indexOf(texto, [indice])	Devuelve la primera posición donde se encuentra el texto buscado, empezando a buscar desde la posición “indice”. Si “indice” no se indica, se toma por defecto el valor 0.
lastIndexOf (texto, [indice])	Busca “hacia atrás” la primera ocurrencia del texto buscado. Indice indica desde qué punto se empieza a buscar “hacia atrás”. Si no se indica el valor de “indice”, se busca desde el final. Si no existe “texto”, devuelve -1.
replace(texto1,texto2)	Busca ocurrencias de la cadena texto1 y las reemplaza por texto2.
split(caracter, [trozos])	Separa la cadena mediante un carácter separador. Trozos indica el máximo de separaciones. Si no se indica, se harán todas las separaciones posibles.
substring(inicio, [fin])	Devuelve la subcadena comprendida entre la posición inicio y la posición fin. Si fin no se indica, se toma como valor el final de la cadena.

Objetos nativos del lenguaje: Math

- Contiene una colección de métodos y constantes.
- Algunas constantes:

Constante	**Valor**
E	Número de Euler
PI	Número Pi
LN2 / LN10	Logaritmo neperiano de 2 / 10
LOG2E / LOG10E	Logaritmo en base 2 / 10 del número de Euler

Objetos nativos del lenguaje: Math

➔ Algunos métodos

Método	**Acción**
floor(numero)	Redondea hacia abajo un número
ceil(numero)	Redondea hacia arriba un número
round(numero)	Redondea al entero más cercano
abs(numero)	Devuelve el número en valor absoluto
max / min(x,y)	Devuelve el mayor / menor de dos números x e y
pow(x,y)	Devuelve x elevado a y
random()	Devuelve un número aleatorio con decimales entre 0 y 1
sqrt(numero)	Devuelve la raíz cuadrada del número indicado

Interacción con el navegador

➔ Algunos objetos:

Objeto	**Función**
location	La dirección actual. Cambiarla para navegar.
document	El documento HTML (DOM).
console	Mostrar información de depuración.
history	El historial de la pestaña actual.
navigator	Información del navegador y acceso a APIs.
screen	La pantalla del dispositivo.
window (tamaño)	El área visible de la pantalla.
alert / confirm / prompt	Diálogos del sistema. Bloquean la página.
localStorage	Almacenamiento persistente en el navegador.
sessionStorage	Almacenamiento durante la sesión.
fetch	Peticiones HTTP.

Interacción con el navegador: URL

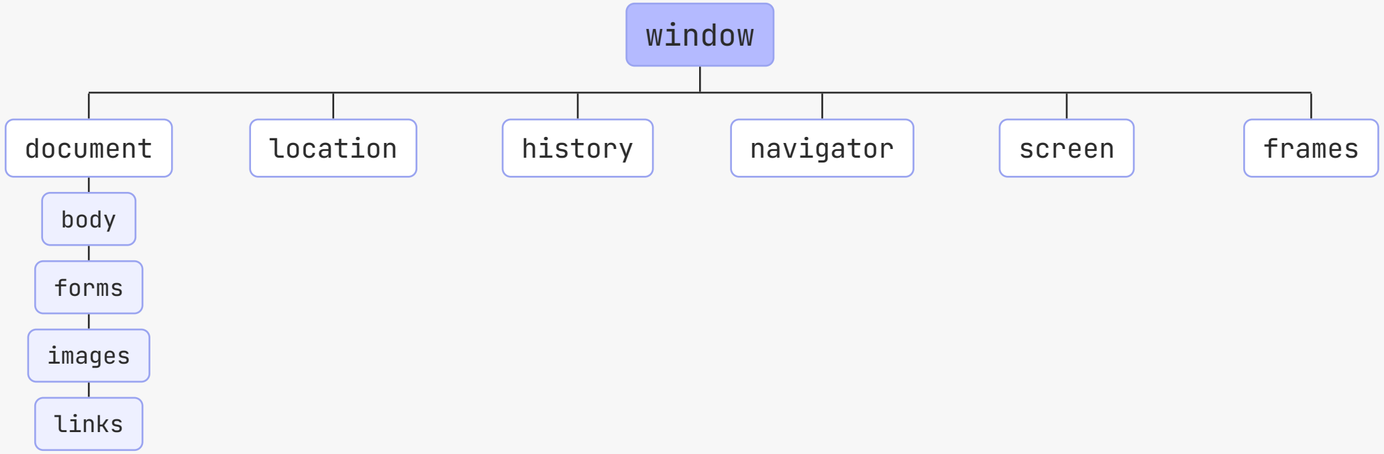
➔ Para trabajar con direcciones (o algunas de sus partes) no es necesario partir las cadenas a mano.

- **new URL(texto)** separa el protocolo, el dominio, la ruta y los parámetros
- **URLSearchParams** lee y modifica los parámetros de consulta codificándolos correctamente.

```
const url = new URL("https://tienda.es/buscar?q=teclado mecánico&page=2");
url.hostname;                          // "tienda.es"
url.searchParams.get("q");             // "teclado mecánico"
url.searchParams.set("page", "3");
url.toString();                        // "https://tienda.es/buscar?q=teclado+mec%C3%A1nico&page=3"

// Cambiar la URL de la página sin recargar (como las SPA)
history.pushState({ pagina: 3 }, "", url.pathname + url.search);
window.addEventListener("popstate", (e) => mostrarPagina(e.state?.pagina ?? 1));   // botón atrás
```

Jerarquía de objetos del navegador



- **window** es el objeto raíz y es implícito: **alert()** equivale a **window.alert()**.
- Los elementos repetidos se agrupan en colecciones con índice desde 0: **window.frames[0]**, **document.forms[0]**.

Propiedades de window

➔ Tamaño y posición

Propiedad	Devuelve
innerWidth / innerHeight	Área visible de la página
outerWidth / outerHeight	La ventana completa, con barras
scrollX / scrollY	Desplazamiento del documento (alias: pageXOffset / pageYOffset)
screenX / screenY	Posición de la ventana en la pantalla

➔ Relación entre ventanas

Propiedad	Devuelve
name	Nombre de la ventana
opener	La ventana que abrió esta
parent	La ventana que contiene este marco
top	La ventana raíz de todos los marcos
self	La propia ventana
closed	true si la ventana se ha cerrado
frames / length	Los marcos que contiene y cuántos son

Métodos de window

Métodos	Para qué sirven
setTimeout / clearTimeout	Ejecutar una función tras un tiempo (ms) / cancelarlo
setInterval / clearInterval	Ejecutar una función cada cierto tiempo / pararlo
open / close	Abrir una ventana nueva / cerrar una ventana
focus / blur	Dar o quitar el foco a la ventana
moveTo / moveBy	Mover la ventana a una posición / desplazarla
scrollTo / scrollBy	Desplazar el documento a unas coordenadas / una cantidad
print	Abrir el diálogo de impresión
getComputedStyle	Obtener los estilos CSS aplicados a un elemento
matchMedia	Evaluar una media query desde JavaScript
requestAnimationFrame	Ejecutar código antes del siguiente repintado
atob / btoa	Decodificar / codificar en Base64

Temporizadores

- **setTimeout** ejecuta una función una vez; **setInterval**, repetidamente.
- Ambos devuelven un identificador para cancelarlos con **clearTimeout** o **clearInterval**.
- El tiempo es un mínimo, no exacto: la función espera a que el navegador esté libre.

```
// Una vez, a los 2 segundos
const id = setTimeout(() => {
  alert("¡Tiempo!");
}, 2000);
clearTimeout(id);   // lo cancela

// Cada segundo, hasta llegar a 10
let segundos = 0;
const reloj = setInterval(() => {
  segundos++;
  if (segundos === 10) clearInterval(reloj);
}, 1000);
```

alert, confirm y prompt

Método	Muestra	Devuelve
alert(mensaje)	Mensaje y botón Aceptar	undefined
confirm(mensaje)	Mensaje, Aceptar y Cancelar	true o false
prompt(mensaje, porDefecto)	Pregunta y campo de texto	El texto escrito, o null si se cancela

Propiedades de document

➔ Estructura

Propiedad	Devuelve
documentElement	El elemento <html>
head / body	Los elementos <head> y <body>
title	El título de la página (se puede cambiar)
activeElement	El elemento que tiene el foco
forms / images / links	Colecciones de formularios, imágenes y enlaces
scripts	Colección de elementos <script>

➔ Información

Propiedad	Devuelve
URL	La dirección completa del documento
domain	El dominio del servidor
referrer	La página desde la que se llegó
lastModified	La fecha de la última modificación
readyState	loading, interactive o complete
characterSet	La codificación (p. ej., UTF-8)
cookie	Las cookies del documento

Métodos de document

Propiedad	Método	Para qué sirve
Buscar	getElementById(id)	El elemento con ese id
	querySelector(selector)	El primer elemento que cumple el selector CSS
	querySelectorAll(selector)	Todos los elementos que lo cumplen
	getElementsByName(nombre)	Los elementos con ese atributo name
Crear	createElement(etiqueta)	Un elemento nuevo
	createTextNode(texto)	Un nodo de texto
Eventos	addEventListener / removeEventListener	Añadir o quitar un manejador de eventos
Flujo	open() / write() / close()	Abrir, escribir y cerrar el flujo del documento

Generación de texto y elementos HTML desde código

➔ En JavaScript, la generación de texto y elementos HTML puede hacerse de tres formas:

- **document.write**: la más antigua y ahora mismo desaconsejada.
- **innerHTML**: interpreta un texto como HTML. Peligroso con datos de usuario.
- **document.createElement** y **elemento.textContent**: la forma segura.

```
<!doctype html>
<html>
  <head>
  </head>
  <body>
    <div id="contenedor">
    </div>
    <script>
      let contenedor = document.querySelector("#contenedor")
      contenedor.textContent = "Esto se escribe desde Javascript"
    </script>
  </body>
</html>
```

Gestión de la apariencia de la ventana

➔ Usando **window** se puede modificar, entre otros parámetros, el tamaño de una ventana.

- **win.resizeBy(width,height):** Cambiar el tamaño de la ventana según el width/height dado en relación con el tamaño actual. Se permiten valores negativos.
- **win.resizeTo(width,height):** Redimensionar la ventana al tamaño dado.
- También existe el evento **window.onresize.**

Parámetros de window.open()

➔ El tercer argumento es una cadena de pares **clave=valor** separados por comas.

Propiedad	**Método**
width / height	Ancho y alto del área de contenido, en píxeles
left / top	Distancia desde el borde izquierdo / superior de la pantalla
popup	Pide una ventana emergente en lugar de una pestaña
scrollbars / resizable	Barras de desplazamiento / permitir redimensionar (yes o no)
menubar, toolbar, location, status	Mostrar u ocultar las barras del navegador (yes o no)
noopener / noreferrer	La nueva ventana no tiene acceso a opener

➔ El tercer argumento es una cadena de pares **clave=valor** separados por comas.

Creación de nuevas ventanas. Comunicación entre ventanas.

- **window.open(url, nombre, opciones)**: abre una ventana o pestaña nueva y devuelve una referencia a ella.
- Los navegadores bloquean las ventanas emergentes que cuyo origen no es un clic del usuario.
- Las ventanas de distinto origen no pueden acceder la una al documento de la otra.

Creación de nuevas ventanas. Comunicación entre ventanas.

➔ Un ejemplo:

```
<script>
    let ventana = window.open(
        "https://jocarsa.com",
        "miVentana",
        "width=800,height=600,left=200,top=100"
    );
</script>
```

Comunicación entre ventanas del mismo origen

➔ **De la principal a la secundaria**

➔ Con la referencia que devuelve **window.open()**:

```
const popup = window.open("popup.html", "pop");
popup.focus();              // traer al frente
popup.miVariable;           // leer una variable
popup.miFuncion();          // llamar a una función
popup.location = "otra.html";
popup.close();              // cerrarla
```

➔ **De la secundaria a la principal**

➔ Con **window.opener**:

```
// En popup.html
const campo = window.opener.document
  .querySelector("#resultado");
campo.value = 42;

window.opener.actualizar();
window.close();
```

➔ Solo funciona si ambas páginas son del mismo origen. Entre orígenes distintos se usa **postMessage**.

Interacción con el usuario

➔ Es posible capturar acciones del teclado o ratón por parte del usuario:

- Ratón:
  - **onclick**: captura un clic simple.
  - **ondblclick**: captura un doble clic.
  - **onmousemove**: captura el movimiento.
- Teclado:
  - **onfocus**: se ha seleccionado el cuadro de introducción de texto.
  - **onblur**: se ha deseleccionado el cuadro de introducción de texto.
  - **keydown**: se ha pulsado una tecla.

Mecanismos del navegador para el almacenamiento y recuperación de información

- Las cookies son pequeños textos que el navegador guarda para un sitio web y envía al servidor en cada petición.
- Se leen y se escriben con **document.cookie**.

```
<script>
  // Guardar
  document.cookie = "nombre=Jose";

  // Leer
  console.log(document.cookie);
</script>
```

Mecanismos del navegador para el almacenamiento y recuperación de información

➔ Para guardar datos que solo necesita el navegador existen:

- **localStorage**: persiste aunque se cierre el navegador.
- **sessionStorage**: se borra al cerrar la pestaña.
- **IndexedDB**: para volúmenes grandes o datos estructurados.

Mecanismos del navegador para el almacenamiento y recuperación de información

➔ Ejemplo **localStorage**:

```
<script>
  // Guardar
  localStorage.setItem("nombre", "Jose");

  // Leer
  console.log(localStorage.getItem("nombre"));
</script>
```

Mecanismos del navegador para el almacenamiento y recuperación de información

➔ Ejemplo **sessionStorage**:

```
<script>
  // Guardar
  sessionStorage.setItem("nombre", "Jose");

  // Leer
  console.log(sessionStorage.getItem("nombre"));
</script>
```